

PuzzleForge -- Phase 3 Final Report

Final Artifact, Evaluation, and Reflection

Project: PuzzleForge -- Agentic Puzzle Game Design and Generation System **Team:** Yixiao Li, Kaizhen Tan, Hanzhe Hong **Track:** A (Technical Build) **Course:** 94-815 Agentic Systems Studio, CMU Heinz College, Spring 2026 **Date:** April 14, 2026

1. Problem Statement

Designing solvable puzzle games is difficult because it requires balancing creativity (interesting level layouts) with correctness (every level must be solvable and avoid deadlocks). A single LLM cannot reliably verify solvability -- it can only guess. This creates a fundamental challenge: the creative generation task and the verification task require different capabilities.

PuzzleForge addresses this by separating creative design from automated verification across 5 coordinated agents in a LangGraph pipeline. The system transforms a user's free-text game concept into a playable, browser-based Sokoban puzzle game where every level is provably solvable by a BFS solver.

2. Architecture

2.1 System Overview

PuzzleForge uses a two-layer architecture with 5 agents:

Agent	Layer	Role	Tools	LLM Tokens
Game Designer	Design	Generate game spec from user concept	LLM (temp 0.8)	Yes
Level Designer	Design	Create/redesign grid layouts	LLM (temp 0.7)	Yes
Developer/Orchestrator	Implementation	Translate designs, route failures	Template engine, routing rules	No (0 tokens)
QA Tester	Implementation	Verify solvability, difficulty, diversity	BFS solver	No (0 tokens)
Debugger	Implementation	Minimal config fixes	LLM (temp 0.3)	Yes

Key design principle: Deterministic where possible, LLM where necessary. Solvability verification, routing classification, and iteration control are handled by code -- not LLM judgment.

2.2 Pipeline Flow

```
User Input --> Game Designer --> Level Designer --> Developer (translate)
--> QA Tester (BFS solver) --> [route_after_qa]:
    all_pass      --> Finalize (playable game)
    has_failures --> Developer (classify & route):
        config_bugs --> Debugger --> Apply Patches --> QA re-test
        design_flaws --> Level Designer (redesign) --> Developer --> QA
re-test
    max_cycles --> Finalize (with failures logged)
    budget/timeout --> Finalize (best available)
```

2.3 Orchestration Framework

Built on **LangGraph StateGraph** with:

- **TypedDict state** shared across all nodes
- **Annotated[List, operator.add]** for append-only trace logging and debug patch accumulation
- **4 conditional edges** encoding routing logic as explicit graph structure
- **9 nodes** in the compiled graph

2.4 Routing Criteria

The Developer classifies QA failures using 10 deterministic rules (no LLM involved):

ID	Criterion	Classification	Routes To
C1-C5	Config bugs (placement issues, count mismatches)	Config bug	Debugger
D1-D5	Design flaws (grid too small, solver timeout, diversity)	Design flaw	Level Designer

Full criteria are implemented in `src/orchestrator/routing.py`.

2.5 Stop Conditions

1. All levels pass QA
 2. 3 debug cycles exhausted
 3. 80K token budget exceeded
 4. 5-minute wall-clock timeout
-

3. Implementation Details

3.1 BFS Sokoban Solver

The ground-truth verification tool (`src/solver/sokoban_solver.py`):

- BFS guaranteeing minimum-move solutions
- State: (`player_position`, `frozenset(box_positions)`)
- Three deadlock types: corner, wall-line, freeze (2x2 cluster)
- 200K state limit to prevent runaway computation
- Additional: `estimate_difficulty()` (1-5 rating) and `level_similarity()` (Jaccard index)

3.2 Inter-Agent Communication

All communication uses Pydantic v2 models. Invalid JSON triggers automatic retry with error feedback. Key schemas: `GameSpec`, `LevelDefinition`, `QAReport`, `DebugPatch`, `RoutingDecision`.

3.3 Game Rendering

HTML/JS template (`src/engine/templates/sokoban.html`) renders playable games with:

- Push and slide mechanics (push moves box 1 tile; slide sends box until it hits a wall or another box)
 - 5 color themes (dark, earthy, icy, gothic, space) driven by `theme_details` in `GameConfig`
 - Emoji-based entity rendering (player, box, target icons chosen by Game Designer)
 - Mechanic badge displayed in the game header
 - Keyboard and touch controls
 - Undo functionality
 - Level progression
 - Win detection
 - **Move-limit enforcement.** Each level carries a `min_moves` value computed by the BFS solver (the shortest possible solution). The UI displays `Moves: X / limit` and a `Best: N` par indicator, where `limit = min_moves + 2`. If the player exceeds the limit without solving the level, the game transitions to a `lost` state and prompts a `reset`. `min_moves` flows from the QA Tester's report into `game_config.levels` during the `finalize` node, so the limit is baked into the rendered HTML.
 - **Tightened difficulty targets.** The Game Designer and Level Designer prompts were hardened to require a meaningful challenge curve: minimum 6x6 grids, 2-box tutorial scaling to 4-5 box expert levels, and layout-driven progression (interior walls, multi-room structures) rather than trivial open rooms.
-

4. Evaluation Results

4.1 Live Pipeline Runs

We ran the pipeline 4 times with different concepts:

Run	Concept	Mechanic	Theme	Levels	Solvability	Rate	Cycles	Tokens	Time	Status
RUN-01	Space station cargo robot	push	space	5	4/5	80%	3	6,093	28s	completed_with_failures
RUN-02	Penguin sliding ice blocks	slide	icy	4	3/4	75%	3	8,538	44s	completed_with_failures
RUN-03	Gothic dungeon gem puzzle	push	gothic	5	5/5	100%	0	2,969	15s	completed
RUN-04	Zero-gravity curling	slide	space	3	2/3	67%	3	8,193	36s	completed_with_failures

Aggregate results:

- **Solvability rate:** 14/17 = 82% (target: $\geq 90\%$) -- slide levels are harder for LLM to design
- **Push solvability:** 9/10 = 90% (meets target)
- **Slide solvability:** 5/7 = 71% (below target -- known limitation)
- **Pipeline completion rate:** 4/4 = 100% (target: $\geq 85\%$)
- **Mean tokens:** 6,448 (well within 80K budget)
- **Mean time:** 30.9s (well within 5-minute timeout)
- **Debug cycles needed:** 0-3 per run (max 3 allowed)
- **Game Designer correctly chose mechanic:** 4/4 (push for warehouse/dungeon, slide for ice/curling)

4.2 Success Criteria Assessment

Criterion	Target	Actual	Status
Solvability rate (push)	$\geq 90\%$	90% (9/10)	PASS
Solvability rate (slide)	$\geq 90\%$	71% (5/7)	NEEDS IMPROVEMENT
Solvability rate (overall)	$\geq 90\%$	82% (14/17)	PARTIAL
Pipeline completion	$\geq 85\%$	100% (4/4)	PASS
Latency	≤ 4 minutes	Mean 30.9s	PASS
Token efficiency	≤ 80 K per run	Mean 6,448	PASS
Routing accuracy	$\geq 75\%$	100% (all failures correctly routed)	PASS

Mechanic selection	Game Designer matches concept	4/4 correct	PASS
Theme diversity	Each run has distinct visual identity	4 unique themes	PASS
Difficulty curve	>= 80% monotonic	4/4 runs monotonic (100%)	PASS

Note on slide solvability: Slide-mechanic levels are harder for the LLM to design correctly because the Level Designer must reason about sliding physics (boxes slide until hitting walls). Push levels achieve 90% solvability while slide levels achieve 71%. The slide-specific design guidance in the Level Designer prompt partially mitigates this, but further improvement is needed (see F-007 in failure log).

4.3 Baseline Comparison

To demonstrate the value of the multi-agent architecture, we ran a **single-LLM baseline**: one GPT-4o call generates all 5 levels (no solver verification, no debug loop, no specialized agents). We then verified baseline outputs with the same BFS solver.

Metric	Single-LLM Baseline	Multi-Agent Pipeline	Improvement
Push solvability	100% (10/10)	90% (9/10)	comparable
Slide solvability	30% (3/10)	71% (5/7)	+41 pp
Overall solvability	65% (13/20)	82% (14/17)	+17 pp
Verified solvable	No (LLM self-report)	Yes (BFS solver)	critical
Tokens per run	1,348	6,448	4.8x (cost of verification)

Key findings:

- Slide mechanics expose LLM spatial reasoning limits.** The baseline achieves only 30% solvability on slide levels -- the LLM cannot reliably reason about sliding physics. The pipeline's debug feedback loop more than doubles this to 71%.
- Push levels are easy for LLMs.** Both approaches achieve high push solvability. The pipeline's value for push is not higher accuracy but **verified correctness** -- baseline levels are unverified and could silently ship broken puzzles.
- The debug loop is the key differentiator.** The pipeline spends ~5x more tokens, but this buys solver verification and iterative repair. For slide levels, this is the difference between 30% and 71% solvability.
- No verification = no guarantees.** The baseline has no way to know which levels are broken. The pipeline catches every unsolvable level and either fixes it or logs it.

4.4 Test Cases Summary

We executed 11 test cases (see `eval/test_cases.csv`):

Category	Tests	Passed	Notes
Happy path / end-to-end	2 (TC-01, TC-05)	2	Full pipeline with debug loop
Solvability & routing	4 (TC-02, TC-04, TC-09, TC-10)	4	Deadlock, routing criteria, escalation,

			timeout
Quality checks	3 (TC-03, TC-06, TC-07)	3	Difficulty, soft diversity, layout similarity
Validation & edge cases	2 (TC-08, TC-12)	2	Pydantic guardrails, slide mechanic selection

Additionally, 15 unit-level tests pass: 5 solver tests, 10 routing criteria tests.

5. Failure Analysis

5.1 Failures Discovered and Fixed

We discovered 5 issues during live pipeline testing (Phase 3), documented in `eval/failure_log.md` (F-004 through F-008):

F-004: Silent Redesign Failure (Severity: HIGH)

What happened: Level Designer's `_redesign_levels()` caught all exceptions with `except Exception: new_levels = []`. When the LLM returned invalid JSON during redesign, the output was silently discarded. The broken level was never replaced, so the pipeline ran all 3 debug cycles doing nothing useful.

Root cause: Overly broad exception handling that prioritized graceful degradation over correctness. The silent failure meant the feedback loop was broken -- QA kept failing, Developer kept routing, but Level Designer's fix was being thrown away.

Fix: Added retry logic with error feedback prompt. On parse failure, a second LLM call is made with the specific error message. Also improved the redesign prompt to include original level dimensions and solver-timeout guidance.

Governance lesson: Silent failure in a feedback loop is worse than a crash. The system appeared to be working (3 cycles completed) but was doing nothing. Observability and explicit error handling are essential for multi-agent reliability.

F-005: Diversity Hard-Blocking (Severity: HIGH)

What happened: When all 5 levels were solvable but one pair had Jaccard similarity > 0.5 , the pipeline treated the diversity violation as a hard failure. It burned all 3 debug cycles trying to fix a cosmetic issue. The Debugger cannot fix diversity (it only adjusts tile positions within a single level), so cycles were wasted.

Root cause: The routing system correctly classified diversity violations as design flaws (D5) and routed to Level Designer. However, the pipeline's iteration logic didn't distinguish between solvability failures (critical) and diversity failures (soft). The `route_after_qa()` function only checked `failed_level_ids` without considering failure severity.

Fix: Added soft-fail logic: if all levels are solvable and at least 1 debug cycle has been attempted, accept the result and finalize. Diversity is now treated as a soft constraint.

Governance lesson: Not all failures are equal. A system that treats cosmetic issues with the same urgency as critical bugs will waste its iteration budget. Severity-aware routing is necessary.

F-006: Debugger Processing Stale Levels (Severity: MEDIUM)

What happened: The Debugger computed `levels_to_fix` by unioning `config_bug_ids` (from the append-only `routing_decisions` list) with `failed_level_ids`. Since `routing_decisions` accumulates across all cycles via `operator.add`, levels fixed in cycle 1 reappeared in cycles 2 and 3.

Root cause: The `operator.add` reducer pattern that works well for trace logging (where you want full history) creates a problem for routing decisions (where you want only current-cycle decisions). The Debugger was reading accumulated state as if it were current state.

Fix: Changed Debugger to use only `failed_level_ids` (set fresh each cycle by `developer_route_node`) instead of unioning with accumulated routing decisions.

Governance lesson: Append-only state is great for observability but dangerous for decision-making. Agents that act on state must distinguish between historical records and current instructions.

5.2 Failure Patterns

Across all 8 documented failures (F-001 through F-008), three patterns emerge:

- 9. **Silent failures are the most dangerous.** F-003 (Pydantic retry) and F-004 (redesign retry) both involved invalid LLM output. F-003 was caught by Pydantic and retried transparently. F-004 was silently swallowed. The difference: explicit validation vs. broad exception handling.
- 10. **Feedback loops need severity awareness.** F-005 showed that a feedback loop without failure severity will treat all issues equally, potentially wasting its iteration budget on low-priority problems while critical issues remain.
- 11. **Append-only state requires careful consumption.** F-006 showed that the same state pattern (append-only lists) that enables good observability can cause incorrect behavior if agents read accumulated history as current instructions.

6. Governance Reflection

6.1 Governance Controls and Their Effectiveness

Control	Implementation	Effectiveness	Evidence
BFS solver as ground truth	QA Tester uses deterministic solver (push + slide), not LLM	Highly effective	14/17 levels verified solvable across 4 runs (push: 9/10, slide: 5/7)

Deterministic routing	10 formal criteria in <code>routing.py</code>	Effective	All routing decisions correct; no misclassification observed
Debugger bounded authority	25% tile change threshold; temp 0.3	Effective	Escalation correctly triggered when needed
3-cycle iteration cap	Hard limit in <code>route_after_qa()</code>	Effective but blunt	Prevents infinite loops but doesn't differentiate failure severity
80K token budget	Checked at conditional edges and in <code>call_llm()</code>	Effective	Never hit in practice (max 6,217 tokens per run)
Pydantic validation	All inter-agent JSON validated	Effective	Caught invalid outputs; retry mechanism works
Append-only trace logging	Annotated[<code>List</code> , <code>operator.add</code>]	Effective for observability	Full history preserved; but required careful consumption (F-006)

6.2 What We Would Change

12. **Severity-aware iteration budget.** Currently, the 3-cycle cap treats all failures equally. A better design would allocate cycles based on failure severity: solvability failures get priority; diversity issues get at most 1 attempt.
13. **Structured error handling instead of broad except.** F-004 showed that `except Exception` in a feedback loop can silently break the entire system. Every exception handler should either retry with feedback, log explicitly, or escalate -- never silently continue.
14. **Separate append-only state from decision state.** `routing_decisions` should be append-only for observability but the Debugger should read a separate `current_cycle_bugs` field that is reset each cycle.

6.3 Human-in-the-Loop

The current system is fully automated from concept to game. Human oversight points:

- **Input:** User provides the game concept
- **Output:** User receives the playable game + full trace log
- **Dashboard:** Streamlit UI allows inspecting every agent's decision, adjusting settings, and re-running

For production use, we would add: routing override (human can reclassify a failure), level editor (human can manually adjust a generated level), and approval gates (human sign-off before finalizing).

7. Lessons Learned

7.1 Technical Lessons

15. **LangGraph's reducer pattern is powerful but requires discipline.** `Annotated[List, operator.add]` elegantly handles trace accumulation, but any agent that acts on accumulated state must be careful to distinguish history from current instructions.
16. **Retry logic is essential for LLM-based agents.** All three LLM agents (Game Designer, Level Designer, Debugger) can produce invalid JSON. The retry-with-error-feedback pattern (send the error back to the LLM) works well and is cheap (~1 extra API call).
17. **The BFS solver is the system's most reliable component.** It never produces false positives or false negatives. Grounding agent output in deterministic verification is the strongest governance mechanism in the system.
18. **Soft vs. hard constraints must be explicit.** The diversity issue (F-005) showed that not distinguishing between "must fix" and "nice to fix" can waste the entire iteration budget.

7.2 Process Lessons

19. **Live testing reveals bugs that unit tests miss.** All 3 Phase 3 bugs (F-004, F-005, F-006) passed unit tests but failed during live pipeline execution. The interaction between agents under real conditions exposed issues that isolated testing could not.
20. **The feedback loop is the hardest part to get right.** The pipeline's forward path (User -> Game Designer -> Level Designer -> Developer -> QA) worked correctly from Phase 2. All Phase 3 bugs were in the feedback loop (redesign, debug, re-test).
21. **Observability pays for itself.** The trace log made it possible to diagnose F-004 (silent redesign failure) by showing that Level Designer was being invoked but producing no changes. Without the trace, this would have been very difficult to find.

8. Individual Contribution Reflection

Yixiao Li

Responsibilities: Game Designer agent, Level Designer agent, BFS solver

Phase 3 contributions:

- Discovered and fixed F-004 (silent redesign failure) by adding retry logic and improved prompts to `level_designer.py`
- Discovered and fixed F-005 (diversity hard-blocking) by adding soft-fail logic to `graph.py`
- Discovered and fixed F-006 (stale level processing) by fixing state consumption in `debugger.py`
- Ran 4 live pipeline tests and collected evaluation data
- Wrote failure analysis and evaluation results

- Updated all evaluation files (test_cases.csv, evaluation_results.csv, failure_log.md, version_notes.md)

What I learned: The most important thing I learned is that multi-agent systems fail in the interaction between agents, not in individual agents. Each agent worked correctly in isolation, but the way they shared state and responded to each other's outputs created emergent bugs. The append-only state issue (F-006) was particularly eye-opening -- a design pattern that works perfectly for logging can cause incorrect behavior when used for decision-making.

9. Evidence Package Summary

Test Cases (11 executed, see `eval/test\_cases.csv`)

- TC-01: Happy path (live run, 5/5 solvable) -- PASS
- TC-02: Solvability loop (deadlock fix) -- PASS
- TC-03: Difficulty curve detection -- PASS
- TC-04: Design flaw routing (D2) -- PASS
- TC-05: Redesign feedback loop (live run) -- PASS
- TC-06: Soft diversity constraint -- PASS
- TC-07: Layout diversity metric -- PASS
- TC-08: Pydantic validation guardrail -- PASS
- TC-09: Debugger escalation threshold -- PASS
- TC-10: Solver timeout routing (D4) -- PASS
- TC-12: Slide mechanic concept (live run) -- PASS

Failure Cases (8 documented, see `eval/failure\_log.md`)

- F-001: Box in corner deadlock (fixed by Debugger)
- F-002: Similar grid layouts (fixed by redesign)
- F-003: Invalid mechanic in GameSpec (fixed by Pydantic retry)
- F-004: Silent redesign failure (fixed: retry logic added)
- F-005: Diversity hard-blocking (fixed: soft-fail logic)
- F-006: Stale level processing (fixed: current-cycle state only)
- F-007: Slide levels harder for LLM to design solvably (partially mitigated: prompt improvement)
- F-008: Level Designer shortfall on slide concepts (known limitation)

Pipeline Run Data (see `eval/evaluation\_results.csv`)

- 4 live API runs (2 push, 2 slide) + 1 sample config verification
- 82% solvability rate across 17 levels (push: 90%, slide: 71%)
- 100% pipeline completion rate
- Game Designer correctly chose mechanic in all 4 runs

10. Future Improvements

- 22. **Severity-aware iteration budget.** Allocate debug cycles based on failure severity rather than treating all failures equally. Solvability failures should get priority; diversity issues should get at most 1 attempt.
- 23. **Difficulty curve enforcement during redesign.** The Level Designer's redesign prompt does not currently enforce difficulty ordering relative to other levels. Adding the full level set's difficulty ratings to the redesign prompt would help maintain monotonic progression.
- 24. **Additional puzzle mechanics.** The HTML template and solver currently support push and slide gameplay. Extending to switch and teleport mechanics would add further variety to the generated puzzles.
- 25. **Human-in-the-loop routing override.** Allow users to reclassify a failure (e.g., override a "design flaw" to "config bug") via the Streamlit dashboard before the next debug cycle.
- 26. **Parallel level generation.** Currently, all levels are generated in a single LLM call. Generating levels independently would allow parallel processing and reduce the impact of a single invalid level on the batch.
- 27. **Solver-guided level design.** Feed solver metrics (states explored, deadlock locations) back to the Level Designer during initial generation, not just during redesign. This could reduce the number of levels that fail QA on the first pass.

Appendix: Repository Structure

```
PuzzleForge/
+-- README.md           # Project overview and setup
+-- AI_USAGE.md         # AI tool usage disclosure
+-- requirements.txt     # Python dependencies
+-- .env.example        # Environment template
+-- docs/
|   +-- final_report.pdf # Phase 3 final report (PDF)
|   +-- architecture_diagram.pdf # System architecture diagram
|   +-- project_summary.pdf # One-page project summary
|   +-- phase3_report.md  # Report source (markdown)
|   +-- phase2_report.md  # Phase 2 report
|   \-- screenshots/     # 9 screenshots with index
+-- media/
|   \-- demo_video_link.txt # 5-minute video link
+-- src/
|   +-- models.py         # Pydantic data models (10 schemas)
|   +-- state.py          # LangGraph state schema
|   +-- config.py         # Configuration and thresholds
|   +-- main.py           # CLI entry point
|   +-- app.py            # Streamlit dashboard
|   +-- agents/           # 5 agent implementations
|   +-- solver/           # BFS Sokoban solver
```

```
|   +-- engine/                # HTML/JS game template engine
|   +-- orchestrator/         # LangGraph pipeline + routing
|   \-- utils/                # Trace logger, LLM wrapper
+-- eval/
|   +-- test_cases.csv        # 11 test scenarios with results
|   +-- evaluation_results.csv # Pipeline + baseline run data
|   +-- evaluation_plan.md    # Evaluation methodology
|   +-- failure_log.md        # 8 documented failures
|   +-- baseline.py           # Single-LLM baseline script
|   +-- baseline_results.json # Baseline run results
|   \-- version_notes.md      # Version history (v0.1-v0.3)
+-- outputs/
|   +-- game.html             # Latest generated game
|   +-- design_doc.json       # Latest design document
|   +-- trace.json            # Latest execution trace
|   +-- run2/                 # Arctic penguin (slide) run
|   +-- run3/                 # Gothic dungeon (push) run
|   +-- run4/                 # Zero-G curling (slide) run
|   \-- sample_traces/        # Phase 2 sample data
\-- phase_submissions/        # Phase archives
```